

Final Project Report

Project Title: Sniff-Then-Spoof ICMP Reply Attack in a Virtualized LAN

Students: Gilbert Ncube & Tinaishe Tapera

Course: Computer and Network Security

1. Abstract

This project demonstrates a real-world network-layer attack in which an attacker intercepts and forges ICMP traffic in a virtual LAN environment. Using Python and Scapy, we implemented a system that sniffs ICMP Echo Requests and immediately injects spoofed Echo Replies. The experiment highlights security weaknesses in ARP and ICMP, showing how unauthenticated protocols can be abused to mislead network hosts. Wireshark was used to capture and analyze legitimate versus spoofed packets. This work directly fulfills the project objectives outlined in the original proposal.

2. Network Topology & Environment Setup

2.1 Virtual Machine Configuration

Machine Role	Operating System	IP Address
Attacker VM	Kali Linux	192.168.56.102
Victim 1 (Sender)	Kali Linux	192.168.56.103
Victim 2 (Target)	Kali Linux	192.168.56.104

- All machines were connected to the **same host-only virtual network**, enabling ARP and ICMP traffic interception.
- No external internet access was required
- Tools used:

- Python 3
- Scapy
- Wireshark

3. Attack Implementation Overview

The attack was implemented in **three stages** using three Python scripts:

1. **ICMP Sniffer** – Captures live ICMP traffic
2. **Manual ICMP Spoofer** – Sends forged ICMP replies
3. **Final Automated Sniff-Then-Spoof Engine** – Performs ARP poisoning, live sniffing, and payload-preserving ICMP spoofing

4. Script Breakdown

4.1 ICMP Packet Sniffer

This script captures and displays all ICMP packets in real time.

```
sniff(iface="eth0", filter="icmp", prn=show_packet)
```

Confirms ICMP traffic between victims

Displays packet structure

Verifies ID, sequence numbers, and payload

4.2 Manual ICMP Spoofing Tool

This script sends a forged ICMP Echo Reply to the victim.

```
ip = IP(src=pretend_to_be, dst=victim_ip)
```

```
icmp = ICMP(type=0)
```

```
packet = ip/icmp
```

```
send(packet)
```

Demonstrates basic spoofing

Victim accepts fake replies from nonexistent hosts

Confirms ICMP has **no authentication**

4.3 Final Automated Sniff–Then–Spoof Attack Script

This script performs:

- ARP poisoning
- Live ICMP interception
- Payload-preserving ICMP spoofed replies

Key features:

- Matches ICMP ID and sequence numbers
- Preserves payload size (fixes “truncated” ping issue)
- Sends fake replies faster than real hosts

This script represents the full **Man-In-The-Middle ICMP spoofing attack**

5. How to Run the Demo (Step-By-Step)

Step 1: Start Wireshark on Attacker VM

On **Kali (Attacker)**:

```
sudo wireshark
```

Select interface:

```
eth0
```

Apply this filter:

```
icmp || arp
```

Step 2: Start the Final Attack Script (Attacker VM)

```
sudo python3 final_attack.py -i eth0 -v
```

Expected output:

```
[*] Starting sniffing on eth0...
```

```
[*] Performing both ARP poisoning and ICMP spoofing
```

Step 3: Start Ping from Victim 1

On **Victim 1 (192.168.56.103)**:

```
ping 192.168.56.104
```

Step 4: Observe Spoofed Replies

Victim 1 will immediately begin receiving replies:

```
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.6 ms
```

```
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=0.7 ms
```

Replies continue even if Victim 2 is shut down

This proves the replies are fake

6. Wireshark Packet Capture & Analysis

Wireshark Filter Used

```
icmp || arp
```

Screenshot 1: ICMP Echo Request from Victim 1

Here apply filter “icmp.type == 8”

The screenshot displays the Wireshark network protocol analyzer interface. The filter bar at the top shows the filter "icmp.type == 8". The packet list pane shows a series of ICMP Echo (ping) requests from source IP 192.168.56.103 to destination IP 192.168.56.104. The sequence numbers in the 'Length Info' column increase by 1 for each request, starting from 1. The packet details pane for frame 42 shows the structure of the ICMP Echo request, including the header checksum, flags, and sequence number (1). The packet bytes pane shows the raw data of the ICMP request.

No.	Time	Source	Destination	Protocol	Length	Info
42	314.076714347	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 43)
44	315.093719206	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=2/512, ttl=64 (reply in 45)
46	316.165734620	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=3/768, ttl=64 (reply in 47)
48	317.168203647	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=4/1024, ttl=64 (reply in 49)
50	318.169573271	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=5/1280, ttl=64 (reply in 51)
52	319.220945531	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=6/1536, ttl=64 (reply in 53)
56	320.229191859	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=7/1792, ttl=64 (reply in 57)
58	321.242766258	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0001, seq=8/2048, ttl=64 (reply in 59)
60	349.300982686	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (reply in 61)
62	350.403532112	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=64 (reply in 63)
64	351.472006239	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=64 (reply in 65)
66	352.526205052	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=4/1024, ttl=64 (reply in 67)
68	353.659314543	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=5/1280, ttl=64 (reply in 69)
72	354.662295947	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=6/1536, ttl=64 (reply in 73)
75	355.691452212	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=7/1792, ttl=64 (reply in 76)
77	356.701502235	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=8/2048, ttl=64 (reply in 78)
79	357.707510409	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=9/2304, ttl=64 (reply in 80)
81	358.807449896	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=10/2560, ttl=64 (reply in 82)
83	359.810098589	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0002, seq=11/2816, ttl=64 (reply in 84)
91	409.994481093	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=64 (reply in 92)
93	411.022294601	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=64 (reply in 94)
95	412.141182548	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=64 (reply in 96)
97	413.144951988	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=64 (reply in 98)
99	414.147020150	192.168.56.103	192.168.56.104	ICMP	98	Echo (ping) request id=0x0003, seq=5/1280, ttl=64 (reply in 100)

Frame 42: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
Ethernet II, Src: PCSSystemtec_72:0d:66 (08:00:27:72:0d:66), Dst: PCSSystemtec_6d:59:10 (08:00:27:6d:59:10)
Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.104
0100 = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
Total Length: 84
Identification: 0xe8d4 (59604)
010. = Flags: 0x2, Don't fragment
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 64
Protocol: ICMP (1)
Header Checksum: 0x5fb4 [validation disabled]
[Header checksum status: Unverified]
Source Address: 192.168.56.103
Destination Address: 192.168.56.104
[Stream index: 6]
Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0xa3e3 [correct]
[Checksum Status: Good]
Identifier (BE): 1 (0x0001)
Identifier (LE): 256 (0x0100)
Sequence Number (BE): 1 (0x0001)

Observed Fields:

- Source IP: 192.168.56.103
- Destination IP: 192.168.56.104
- ICMP Type: 8 (Echo Request)
- Sequence Number: increments normally

This confirms the legitimate ping request.

Screenshot 2: Spoofed ICMP Echo Reply from Attacker

Here apply filter “icmp.type == 0”

The screenshot displays the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The filter bar at the top shows the active filter: `icmp.type == 0`. The packet list pane shows a series of ICMP Echo (ping) replies from source IP 192.168.56.104 to destination IP 192.168.56.103. The selected packet (No. 43) is expanded in the packet details pane, showing the following structure:

- Frame 43: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
- Ethernet II, Src: PCSSystemtec_6d:59:10 (08:00:27:6d:59:10), Dst: PCSSystemtec_72:0d:66 (08:00:27:72:0d:66)
- Internet Protocol Version 4, Src: 192.168.56.104, Dst: 192.168.56.103
 - 0100 = Version: 4
 - 0101 = Header Length: 20 bytes (5)
 - Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
 - Total Length: 84
 - Identification: 0x6e12 (28178)
 - 000. = Flags: 0x0
 - ...0 0000 0000 0000 = Fragment Offset: 0
 - Time to Live: 64
 - Protocol: ICMP (1)
 - Header Checksum: 0x1a77 [validation disabled]
 - [Header checksum status: Unverified]
 - Source Address: 192.168.56.104
 - Destination Address: 192.168.56.103
 - [Stream index: 6]
- Internet Control Message Protocol
 - Type: 0 (Echo (ping) reply)
 - Code: 0
 - Checksum: 0xab03 [correct]
 - [Checksum Status: Good]
 - Identifier (BE): 1 (0x0001)
 - Identifier (LE): 256 (0x0100)
 - Sequence Number (BE): 1 (0x0001)

The packet bytes pane on the right shows the raw data of the selected packet, including the Ethernet II header, IP header, and ICMP Echo (ping) reply payload.

Observed Fields:

- Source IP: 192.168.56.104 (spoofed)
- Destination IP: 192.168.56.103
- ICMP Type: 0 (Echo Reply)
- Same Sequence Number as request
- Same payload length

This confirms the attacker successfully forged a valid reply.

Screenshot 3: ARP Poisoning Packet

Here apply filter “eth.src == 08:00:27:38:30:c6 && arp.opcode == 2” for ARP replies sent by attacker MAC

The screenshot displays a Wireshark capture of network traffic. The filter applied is `eth.src == 08:00:27:38:30:c6 && arp.opcode == 2`. The packet list shows several ARP replies from the source MAC `PCSSystemtec_38:30:c6` to the destination MAC `PCSSystemtec_72:0d:66`. The packet details pane for the selected packet (Frame 138) shows the following structure:

- Frame 138: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
- Ethernet II, Src: PCSSystemtec_38:30:c6 (08:00:27:38:30:c6), Dst: PCSSystemtec_72:0d:66 (08:00:27:72:0d:66)
- Destination: PCSSystemtec_72:0d:66 (08:00:27:72:0d:66)
- Source: PCSSystemtec_38:30:c6 (08:00:27:38:30:c6)
- Type: ARP (0x0806)
- [Stream index: 15]
- Padding: 00000000000000000000000000000000
- Address Resolution Protocol (reply)
 - Hardware type: Ethernet (1)
 - Protocol type: IPv4 (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: reply (2)
 - Sender MAC address: PCSSystemtec_38:30:c6 (08:00:27:38:30:c6)
 - Sender IP address: 192.168.56.145
 - Target MAC address: PCSSystemtec_72:0d:66 (08:00:27:72:0d:66)
 - Target IP address: 192.168.56.103

The packet bytes pane shows the raw hex and ASCII data of the packet, including the Ethernet II header, Internet Protocol Version 4 header, and ARP data.

Observed Fields:

- ARP Operation: Reply (op = 2)
- Fake MAC mapped to victim IP

This confirms the attacker became Man-In-The-Middle. Here the attacker:

- Listens for ARP "who-has" requests.
- Sends forged ARP replies claiming that the requested IP is at the attacker's MAC address (08:00:27:38:30:c6).
 - **Spoofed packets:** Source IP doesn't match any real machine

7. Security Analysis

This experiment demonstrates several critical weaknesses:

Vulnerability	Demonstrated
ICMP has no authentication	✓
ARP has no trust verification	✓
MITM possible on LAN	✓
Source IP spoofing	✓
Victims blindly trust replies	✓

Without firewall rules, IDS, or encrypted tunnels, a local attacker can fully manipulate ICMP communication.

8. Limitations

- Works only on local Layer-2 networks
- Does not bypass encrypted protocols
- Easily detectable by IDS/IPS
- ICMP filtering would block this attack

9. Conclusion

This project successfully demonstrates a complete sniff-then-spoof ICMP attack in a controlled virtual environment. By exploiting weaknesses in ARP and ICMP, an attacker was able to intercept live traffic and inject forged Echo Replies that were trusted by the victim machine.

Through Wireshark analysis, we confirmed that spoofed packets matched legitimate traffic fields, proving that unauthenticated network-layer protocols remain vulnerable. This project highlights why modern networks require intrusion detection, packet filtering, and encryption for protection.